Sprint 3 – Technical_Report.pdf

Team 8's Submission for Sprint #3

Team Name: Wolf of The 7th Street

Pipeline Contract and Stage Map

- **Pipeline Contract:**

1. **Project question addressed by this pipeline:** Our question is if there are any trends from our analysis that we can use to determine an accurate price prediction. Our analysis_text.sql and analysis_parquet.sql pipelines address the given question.
2. **How result contributes to final project :** The results help contribute to the final project by helping us calculate the monthly stock-price increase counts, total comparable records, and ratio of increases.
3. **Raw input file, delimiter, header status, and fields used:**

   Raw input file: TMCV_minute.tsv

   Delimiter: Tabs

   Header status: name date open high low close volume price_change volume_change

   Field used: name, date, open, high, low, close, volume, price_change, volume_change

4. **Expected schema of analysis-ready intermediate table:** Expected to be a .tsv file with a header row and four columns:

Column name	**Data type**	**Description**
symbol	VARCHAR	Stock ticker symbol
timestamp	VARCHAR	Date and time of stock record
close	DOUBLE	Closing stock price
price_change	VARCHAR	Price movement (Increased/Decreased)

5. **Output schemas for shell, DuckDB-text, and DuckDB-Parquet**: All three output schemas are expected to be .tsv files with the same four columns being used:

Column name	**Date type**	**Description**

month	Integer	Month number (1-12)
increase_count	Integer	Number of records that increased that month
compared_rows	Integer	Total records that increased or decreased during the month
Increase_ratio Decimal	Decimal format	increase_count / compared_rows, four decimal place

6. **Rules:**

   Filtering: Only rows where the price_change has increased or decreased are included.

   Missing-value: None rows don't contribute to the analysis stages but are accepted as a valid price-change in the preparation stage.

   Grouping: The scripts extracts month from timestamp, group records from month number, and calculate the increase count, comparable row count, and increase ratio.

   Ordering: DuckDB sorts months in ascending order. Shell script loops through months in ascending order and ignores months with no comparable records.

   Ranking: Contains monthly aggregates instead of ranked individual records

   Tie-handling: Same as ranking.

- **Stage Map:**

  Raw input

  -> Validate and normalize input

  -> Analysis-read TSV

  -> Shell-text analysis

  -> DuckDB-text analysis

  -> DuckDB-text results

  -> Build Parquet file

  -> DuckDB-Parquet analysis

  -> DuckDB-Parquet results

  -> Equivalence check and timing evidence

-> One scheduled partition per day (not implemented yet)

-> Final project analysis artifact

Parquet Creation Process

- **Original-text:** The original file is ‘work/analysis_ready.tsv’ and is a tab-separated file with four columns (symbol timestamp close price_change)
- **Parquet creation process:**
  We used ‘build_parquet.sql’ with DuckDB. Reading ‘work/analysis_ready.tsv’ using ‘read_csv’, the script specifies the tab delimiter and header row with expected data types. The processed data goes into ‘work/analysis_ready.parquet’ using DuckDB’s parquet format.

Shell, DuckDB-text, and DuckDB-Parquet implementation

- **Shell implementation:**

  Script: ‘pa3/run_shell_analysis.sh’

  Choice: Reads ‘work/analysis_ready.tsv’, validates it and calculates the monthly price-increased statistics using Grep, AWK and other text processing commands taught in class. The results are saved in ‘out/shell_result’.

- **DuckDB-text implementation:**

  Script: ‘sql/analysis_text.sql’

  Choice: Reads ‘work/analysis_ready.tsv’, keeps increased and decreased records by months. Counts increases and comparable rows to calculate ‘increase_ratio’ to order the results by month in ‘out/duckdb_text_result.tsv’.

- **DuckDB-Parquet implementation:**

  Script: ‘sql/analysis_parquet.sql’

  Choice: Reads ‘work/analysis_ready.parquet’ as a structured parquet file, keeps increased and decreased records by months. Counts increases and comparable rows to calculate ‘increase_ratio’ to order the results by month in ‘out/duckdb_parquet_result.tsv’.

Three-Way Equivalence

- **Shell vs DuckDB-text vs DuckDB-Parquet inside pa3/**

  Shell: work/analysis_ready.tsv → out/shell_result

DuckDB-text: work/analysis_ready.tsv → out/duckdb_text_result.tsv

DuckDB-parquet: work/analysis_ready.parquet → out/duckdb_parquet_result.tsv

All three analysis paths use different methods of execution but implement the same logic. When comparing all their outputs individually and looking in ‘out/result_comparison’, all three analysis paths agree and produce the same readable results.

Benchmark Results and Interpretation

- **Results:** The benchmark used 1,033,587 input records. The original TSV file was 49,605,893 bytes, while the equivalent Parquet file was 7,536,646 bytes. Therefore, the Parquet file was approximately 84.8% smaller than the original TSV file.

  Each processing path was run three times. The shell implementation took 6.375, 6.325, and 6.316 seconds, with an average of approximately 6.339 seconds. DuckDB running on the original TSV file took 0.372, 0.320, and 0.317 seconds, with an average of approximately 0.336 seconds. DuckDB running on the Parquet file took 0.137, 0.133, and 0.137 seconds, with an average of approximately 0.136 seconds. All nine runs were verified successfully, and all three processing paths produced equivalent results.

- **Interpretation:** For this dataset and analysis, DuckDB over Parquet was the fastest processing path. It was approximately 2.5 times faster than DuckDB over the original text file and approximately 47 times faster than the shell implementation. The DuckDB-Parquet results were also consistent across the three runs, varying only slightly between 0.133 and 0.137 seconds.

  The smaller Parquet file size likely contributed to the faster DuckDB query time because less data had to be read and parsed. However, the benchmark may have been affected by system load, file-system caching, and process startup time. These results describe this specific dataset and query and should not be interpreted as proof that Parquet or DuckDB is universally faster for every workload.

  The DuckDB queries were also easier to maintain because the filtering, grouping, and ratio calculation were expressed clearly in SQL. Based on the smaller storage size, faster query time, consistent results, and maintainability, the evidence supports using DuckDB with Parquet for the next stage of the project.

Background-run Evidence

- **Launch command:** bash pa3/scripts/background_exe.sh
- **Evidence file:**
- # Launched Command
- nohup "scripts/run_pipeline.sh pa3/data/sample/sample.tsv" > "logs/background_run_log" 2>&1 &
-

- ## PID & JOB
- PID: 983121
- Job: [1]+ 983121 Running nohup bash -c "$CMD; echo \$? > logs/exit_code.txt" > "$LOG" 2>&1 &
-
- ## PS Snapshot
- PID PPID STAT ELAPSED CMD
- 983121 983114 S 00:01 bash -c scripts/run_pipeline.sh pa3/data/sample/sample.tsv; echo $? > logs/exit_code.txt
-
- ### log tail
- nohup: ignoring input
- [2026-10-25 17:09:07] Input: /mnt/scratch/CS131_jelenag/projects/team08_sec2_Fall2026/scripts/7thStWolf-Nifty100-Stock/pa3/data/sample/sample.tsv
- [2026-10-25 17:09:07] Stage 1/5: prepare and validate analysis-ready TSV
- [2026-10-25 17:09:39] Stage 2/5: run shell analysis
- [2026-10-25 17:09:45] Stage 3-4/5: Building Parquet & Run DuckDB Anaylsis
- [2026-10-25 17:09:46] Stage 5/5: compare results
- [2026-10-25 17:09:47] All three analysis paths agree
- [2026-10-25 17:09:47] Pipeline completed successfully
-
- ### Completeion Evidence
- Exit Status: 0

Meaningful Partition Design and Scheduled-run Results

- **Partition Design:** Dataset was partitioned into Parquet Files by Stock Name. To converse space, we decided not to create all possible partitions, as there are 100 stocks. Instead, for this sprint we chose 4 random stocks and partitioned them, to execute the scheduled run.
- **Scheduled-run Results:** The four selected stock partitions TATACONSUM, TATAPOWER, ULTRACEMCO, and ZYDUSLIFE were processed successfully. Each scheduled run processed one partition, generated its analysis output, and completed with exit status 0. After all four partitions were processed, the scheduler confirmed that no incomplete partitions remained.

Failures Encountered and Diagnosis

- **One Failure Encountered:** The main problem encountered during the sprint was path conflicts. The pipeline was required to run from the `pa3` directory, but different scripts used relative paths for inputs, outputs, logs, SQL files, and Parquet files. As a result, some commands could not find the expected files when they were run from a different directory.
- **Team Resolution and Diagnosis:** The issue was diagnosed by reviewing the error messages and checking the current working directory used by each script. The team updated the scripts to establish the `pa3` directory as the working directory and used script-relative paths for files such as `work/analysis_ready.tsv`, `out/`, `logs/`, and the SQL files. After these changes, the

pipeline, background execution, and scheduled partition scripts ran successfully from the required project location.

Team Contributions and AI-use Disclosure

- **Team Members:**
  - Aydan Kaneshiro – Product Manager (Sprint 3)
  - Devarsh Shroff – Data Engineer (Sprint 3)
  - Rishil Nilesh Patel – Data Engineer (Sprint 3)
  - Vakeesan Moorthy – Data Engineer (Sprint 3)
  - Jeremy Lim – Data Storyteller (Sprint 3)
- **Team Contribution:**
  - **Project Team Board:** https://docs.google.com/spreadsheets/d/1YEvWa5adA3sNDbaax59ER2YFxVO7ba8ewUgEzhEdeAg/edit?gid=0#gid=0
  - **Branches Used in Sprint 3:** feature/pa3-analysis-pipeline
- **AI-use Disclosure:**
  - Claude was used for debugging errors in the code and github commands.